# From predicted loss to a validated capability forecast

This companion expands notes §§2, 9–12 and blog §§2, 9–12. A good next-token-loss fit does not establish that a target will pass a task, remain stable or meet a deployment budget.

## Why accuracy loses information

Consider an illustrative two-choice task. Two models each assign probability 0.30 to the correct answer, so their correct-answer NLL is identical. One assigns 0.40 to the distractor, the other 0.20, with remaining mass elsewhere. Ranked argmax choices differ despite identical correct-answer loss. This illustrates why correct-answer NLL alone need not determine accuracy; task formatting and distractor probabilities matter. Schaeffer et al., downstream predictability.

For fixed evaluation text, BPB measures continuous changes before a discrete correctness threshold changes. It is useful only if its relationship to the final metric is empirically calibrated for the task and model family.

## What AI2's two-stage fit actually does

AI2 first fits size and tokens to an intermediate loss, then calibrates intermediate loss to ranked-classification accuracy. Its first stage uses final ladder measurements; its second stage also uses intermediate checkpoints. The paper uses different intermediate features and finds task-dependent prediction performance. Checkpoint abundance does not make those points independent training runs. Bhagia et al., v2, §§2–4.

A two-stage predictor is

$$\hat{a}(N, D) = g_\phi(\widehat{L}_\theta(N, D)).$$

To assess it, test both the loss extrapolation and the calibration, then the composed prediction. A calibration tested using actual target loss bypasses stage-one error and is not the same test as the full forecast.

## Difficulty clustering is another experiment

COD groups questions by pass-rate signatures across small models, fits informative difficulty subsets and maps their prediction back to the full benchmark. It addresses tasks where aggregate small-model accuracy is near zero. Cluster choices, sampling budgets and subset-to-full calibration are learned components: freeze them before final holdout evaluation. Mixture or architecture changes can alter which questions are difficult. COD, v4.

## Four distinct validation gates

| Gate | Quantity tested | An informative failure |
|---|---|---|
| Extrapolation | Loss or candidate-gap prediction | Residual exceeds expected noise |
| Capability | Task floors and ranking | Continuous proxy fails to predict task outcome |
| Numerical stability | Long-horizon production update | Divergence or growing layer/router imbalance |
| Systems | Time, memory and recovery | Quality gain costs too much wall-clock time |

Ultra reports an output-gradient precision change associated with divergence and a rollback/restoration of FP32 accumulation that re-stabilized that segment. A later failure required an annealing intervention. Small-scale loss fits alone would not certify these production decisions. Nemotron 3 Ultra, §2.7; case study.

## Preserve the meaning of a holdout

Before evaluating it, fix a tolerance, capability floor and procedure for inconclusive results. Once the result changes the recipe or fitting procedure, that run becomes development evidence. The revised recipe needs fresh confirmation.

For staged runs, record curriculum, context extension, checkpoint averaging and standardized post-training probes. A changed endpoint or post-training pipeline changes what the forecast is about. Archive the run/branch lineage and original decisions so a later recipe revision does not silently rewrite the evidence.

**Required artifact:** stage-one and composed forecast errors, task calibration, stability/throughput measurements and a dated acceptance record. See the validation protocol and execution ledger.

Back to the blog reading guide · Quarto notes